

1. Introduction

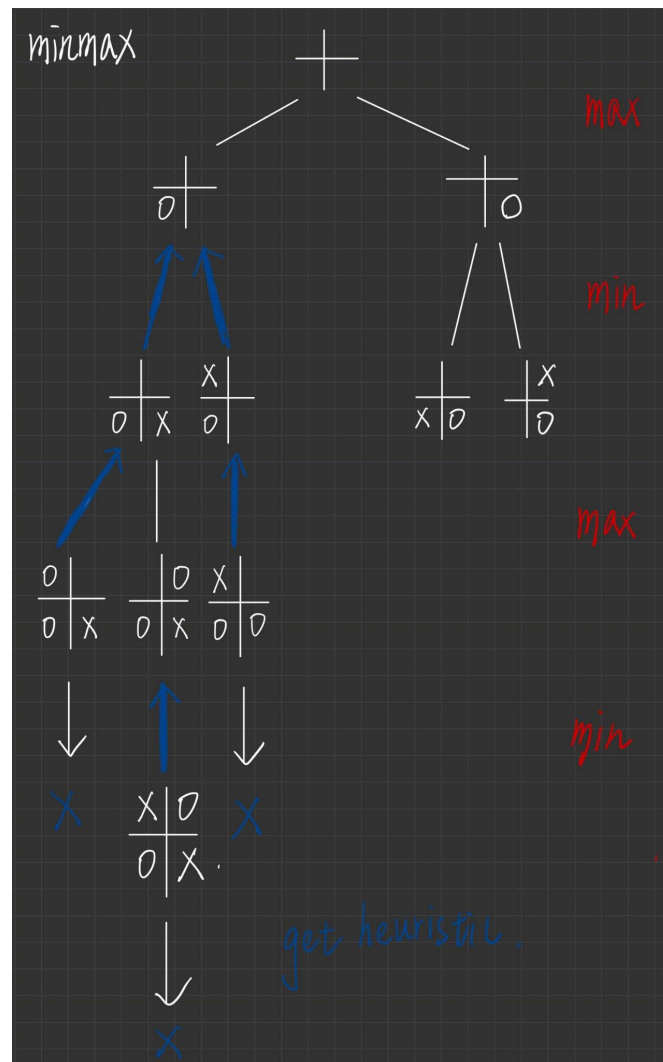
In this report, I will show how I develop and implement advanced adversarial search techniques for the game Connect Four.

2. Implementation

a. Minimax Search

- Explain the algorithm

- I. I drew a simple graph to illustrate how the recursive `minimax()` function works. The `minimax()` function is called whenever there is still space on the board for the player to make a move. When the game reaches a terminal state (win, loss, or draw), or when the search reaches the specified depth, the function calls `get_heuristic()` to evaluate the current board state



II. After evaluation, the scores are propagated back layer by layer through recursion. Depending on which

player's turn it is, the function keeps track of either the maximum or minimum score. If it's our player's turn (Maximizing Player), it chooses the maximum value among all possible moves. On the other hand, if it's the opponent's turn (Minimizing Player), it selects the minimum value.

III. The `get_heuristic()` function calculates the number of "two-in-a-row" and "three-in-a-row" combinations for both our player and the opponent. It also checks if either player has already won. The function returns an overall score that simulates human intuition in evaluating board states. This heuristic score is essential because it provides the base case score that initiates the backpropagation of values through the recursive layers.

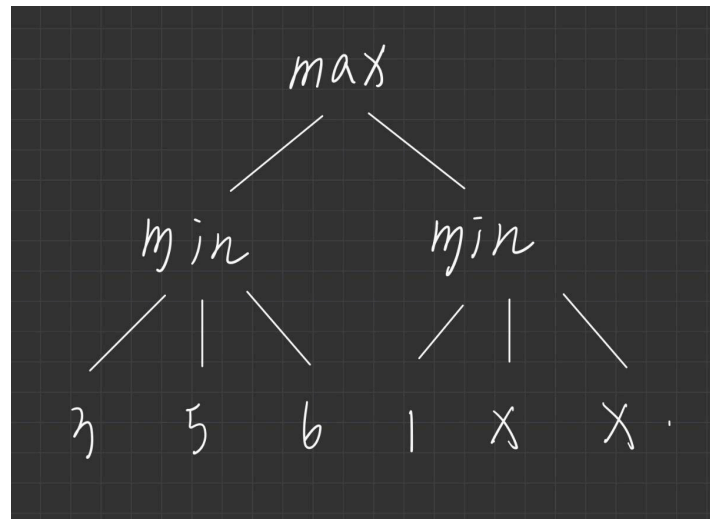
- Results & Evaluation

```
=====
Game 100/100 finished.
execute time 748905.79 ms
Summary of results:
P1 <function agent_minimax at 0x119426480>
P2 <function agent_reflex at 0x1194265c0>
{'Player1': 100, 'Player2': 0, 'Draw': 0}
=====
DATE: 2025/04/04
STUDENT NAME: 王子儀
STUDENT ID: 111705068
=====
```

b. Alpha-Beta Pruning

- Explain the optimization

I. I drew a simple graph to explain how it works.



As you can see, since we the function will travel through all the possible steps, for example, since now the alpha is 3, and then the function calls alphabeta to search for the right tree. Since the first node is 1, 1 is smaller than 3, it will break. Because in the top level, we want to maximize the value, definitely won't pick this tree.

II. In my code, when

```
if beta <= alpha:
    break
```

The search for this tree will stop.

- Results & Evaluation

```
=====
Game 100/100 finished.
execute time 221706.64 ms
Summary of results:
P1 <function agent_alphabeta at 0x10e7fa340>
P2 <function agent_reflex at 0x10e7fa3e0>
{'Player1': 94, 'Player2': 6, 'Draw': 0}
=====
DATE: 2025/04/04
STUDENT NAME: 王子儀
STUDENT ID: 111705068
=====
```

As you can see, the cost of time has declined.

c. Stronger AI Agent

- Techniques Used

To develop agent_strong(), I designed a custom heuristic function and applied it within a standard Alpha-Beta search

algorithm (depth = 4). The agent uses the same search structure as the Alpha-Beta agent, but with an improved heuristic evaluation designed to make more strategic decisions.

The key optimizations implemented include:

- A stronger heuristic function that prioritizes winning opportunities and blocking threats.
- Board control strategies, particularly favoring central column placements.
- Using forced win detection and threat suppression, simulating human-style intuition.
- Monte Carlo Sampling or Probabilistic Search: Introduce simulations or rollout-based evaluations to enhance strategy.

This allowed the agent to outperform the baseline Alpha-Beta agent by a noticeable margin, especially in situations requiring both offense and defense within a few turns.

- Advanced Heuristic Function
 - I. Winning Potential (Forced Wins)
If a win is achievable immediately, the function returns a very large value to force the agent to take it.
 - II. Defensive Play (Blocking Opponent)
If the opponent is one move away from winning, the function heavily penalizes inaction, pushing the agent to block.
 - III. Board Control (Center Dominance)
The center column is preferred because it allows more potential winning combinations.
- Results & Evaluation
I can't beat Alphabeta !!!

```
=====
Game 100/100 finished.
execute time 309171.28 ms
Summary of results:
P1 <function agent_alphabeta at 0x1076223e0>
P2 <function agent_strong at 0x107622660>
{'Player1': 83, 'Player2': 17, 'Draw': 0}
=====
DATE: 2025/04/04
STUDENT NAME: 王子儀
STUDENT ID: 111705068
=====
```

3. Analysis & Discussion

- Difficulties in Designing a Strong Heuristic

One of the most challenging parts was finding a good balance between offensive and defensive heuristics. A naive scoring system might overly favor immediate threats or greedily chase connections without considering future consequences.

Fine-tuning the weights for two-in-a-row and three-in-a-row patterns, while ensuring proper scaling of win/loss detection, required multiple iterations and testing.

- Weaknesses of agent_strong()

While agent_strong() performs well overall, it still has some limitations: High computation time at deeper levels, since Alpha-Beta pruning is used but not optimized for move ordering.

- How to Improve agent_strong()

Several improvements could be made:

1. Move Ordering Optimization: Sorting moves (e.g., center-first or by heuristic score) can increase pruning efficiency.
2. Dynamic Depth Adjustment: Increase search depth in critical situations (e.g., when a win is near).

4. Conclusion

In this project, I developed and evaluated a Connect Four AI agent using the Minimax and Alpha-Beta Pruning algorithms, with an enhanced

version, `agent_strong()`, that incorporated a more advanced heuristic for move evaluation.

Key Achievements:

- Alpha-Beta Pruning was successfully implemented to optimize the Minimax search, significantly reducing the computational cost of evaluating deep game trees while maintaining correct decision-making.
- The `get_heuristic_strong()` function was improved to prioritize winning potential, defensive play, and board control—strategies that were essential for its superior performance compared to the baseline Alpha-Beta agent.

Future Enhancements:

To further improve the performance of `agent_strong()`, I plan to explore:

- Move ordering optimizations to enhance pruning efficiency.
- Dynamic depth adjustment based on the state of the game.

This homework is pretty interesting, I wish I could spend more time on it to beat Alphabeta.